شرح دالة `_newestFirst` سطرًا بسطر

تطبيق ASEC · الملف: **`video_remote_data_source.dart`**

`lib/features/video/v1_videos/data/data_source/remote_data_source/`

---

0 الدالة كاملة

```
List<VideoItem> _newestFirst(Iterable<VideoItem> videos) {
  final List<VideoItem> list = videos.toList()
    ..sort((VideoItem a, VideoItem b) => (b.createdAt ?? DateTime(0))
        .compareTo(a.createdAt ?? DateTime(0)));
  return list;
}
```

وظيفتها في جملة واحدة: تأخذ مجموعة فيديوهات غير مرتّبة، وتُعيدها قائمة مرتّبة **الأحدث أولًا**.

1 السطر الأول — توقيع الدالة

```
List<VideoItem> _newestFirst(Iterable<VideoItem> videos) {
```

- `_newestFirst` — الشرطة السفلية `_` في أول الاسم تعني أن الدالة **خاصة** بهذا الملف: لا يستطيع أي ملف آخر استدعاءها.
- تستقبل `Iterable<VideoItem>` وليس `List`. و`Iterable` نوع أوسع وأكثر مرونة: ناتج `snap.docs.map(VideoModel.fromDoc)` من هذا النوع، وهو **كسول** (lazy) — لم يُنشئ العناصر بعد، لكنه يعرف كيف يُنشئها عند الطلب.
- وتُعيد `List<VideoItem>` حقيقية ومرتّبة.

2 `videos.toList()`

```
final List<VideoItem> list = videos.toList()
```

يُنشئ قائمة حقيقية قابلة للتعديل في الذاكرة، ولذلك سببان:

1. النوع `Iterable` لا يملك الدالة `sort()` أصلًا — الترتيب متاح على `List` فقط.
2. هنا تُنفَّذ `map()` الكسولة فعليًا، فيُبنى كائن `VideoItem` واحد لكل مستند قادم من Firestore.

3 العامل `..` — التسلسل (cascade)

معناه باختصار: **نفّذ هذه الدالة على الكائن نفسه، ثم أعِد لي الكائن — لا ما أعادته الدالة.**

الدالة `sort()` تُعيد `void`، أي لا تُعيد شيئًا. لذلك بنقطة واحدة ينكسر الكود: يصبح `list` قيمته `void` بدل أن يكون القائمة.

```
final list = videos.toList().sort(...);   // wrong
final list = videos.toList()..sort(...);  // right
```

السطر الأول خطأ، والثاني صحيح.

> **ملاحظة مهمة:** `sort()` ترتّب القائمة *في مكانها* (in place) ولا تُنشئ قائمة جديدة. فـ `list` هو الكائن نفسه، بعد أن أصبح مرتّبًا.

4 دالة المقارنة — كيف يقرّر الترتيب؟

```
(VideoItem a, VideoItem b) => (b.createdAt ?? DateTime(0))
    .compareTo(a.createdAt ?? DateTime(0))
```

الدالة `sort` تُناولك فيديوين، `a` و `b`، وتسألك: أيّهما يأتي أولًا؟ وأنت تُجيب برقم:

القيمة المُعادة	المعنى
رقم سالب	`a` يأتي أولًا
رقم موجب	`b` يأتي أولًا
`0`	متساويان

والدالة `compareTo` على `DateTime` تفعل هذا تمامًا: `x.compareTo(y)` تكون سالبة عندما يكون `x` **أقدم** من `y`.

5 لماذا `b` قبل `a`؟

انتبه للترتيب داخل المقارنة: مكتوب `b.compareTo(a)` وليس `a.compareTo(b)`. وهذا **مقلوب عن قصد**.

المقلوب يعني ترتيبًا **تنازليًا**، أي الأحدث أولًا — وهو المطلوب في قائمة الدروس. ولو عكستَهما لحصلتَ على الأقدم أولًا.

6 العامل `??` والقيمة `DateTime(0)`

```
b.createdAt ?? DateTime(0)
```

العامل `??` يعني: **إذا كان ما على اليسار `null` فاستخدم ما على اليمين.**

وقيمة `createdAt` قد تكون `null` فعلًا: عند إضافة فيديو نكتب الحقل `created_at` بالقيمة `FieldValue.serverTimestamp()`، ويملؤها خادم Firestore لا الجهاز — فتمرّ لحظة تكون فيها النسخة المحلية بلا تاريخ. وبدون `??` سينهار التطبيق عند محاولة `null.compareTo(...)`.

أما `DateTime(0)` فهو العام صفر، أي **أقدم تاريخ ممكن**. لذلك يُعامَل الفيديو الذي لم يصله تاريخ بعد على أنه قديم جدًا فيهبط إلى آخر القائمة، بدل أن يُسقط الشاشة كلها.

7 السطر الأخير

```
return list;
```

يُعيد القائمة بعد أن رُتّبت. وبما أن `sort` عدّلت القائمة في مكانها، فهذه هي القائمة نفسها التي أنشأناها في السطر الثاني.

8 الدالة نفسها بالطريقة المطوّلة

سلوك مطابق تمامًا، لكن بلا اختصارات — اقرأها إن بدا السطر الأصلي مزدحمًا:

```
List<VideoItem> _newestFirst(Iterable<VideoItem> videos) {
  final List<VideoItem> list = videos.toList();   // 1. قائمة حقيقية

  list.sort((VideoItem a, VideoItem b) {          // 2. رتّبها
    final DateTime dateA = a.createdAt ?? DateTime(0);
    final DateTime dateB = b.createdAt ?? DateTime(0);
    return dateB.compareTo(dateA);                // الأحدث أولًا
  });

  return list;                                    // 3. أعِدها
}
```

9 مثال عملي

ثلاثة فيديوهات في قاعدة البيانات:

الفيديو	تاريخ الإنشاء
الدرس 1	1 يناير
الدرس 2	5 مارس
الدرس 3	`null` — ما زال يُحفظ

الناتج بعد `_newestFirst` :

> **الدرس 2** (5 مارس) ← **الدرس 1** (1 يناير) ← **الدرس 3** (عُومل كأنه العام صفر، فنزل إلى الأسفل).

10 لماذا نرتّب داخل التطبيق لا في Firestore؟

لأن الاستعلام `.where('published', isEqualTo: true).orderBy('created_at')` يحتاج **فهرسًا مركّبًا** (composite index) في Firestore، ولا يعمل قبل نشره.

والقائمة هنا صغيرة، فترتيبها على الجهاز لا يكلّف شيئًا ويُغنيك عن نشر أي فهرس — ولهذا جاء الملف `firestore.indexes.json` فارغًا.

النمط نفسه مكرّر في ملفين آخرين: `_newestFirst` في `quiz_remote_data_source.dart` للاختبارات، و `watchMembers()` التي ترتّب بالاسم بدل التاريخ.
